

ICL 自动数据标注设计说明

竞赛： FlagOS 开放计算全球挑战赛
项目：长上下文场景中大模型自动数据标注（赛道三）

提交物清单

序号	文件	说明
1	ICL 自动数据标注设计说明.pdf	说明文档，涵盖方案阐述、架构设计和算法详解等
2	submission_results.zip	已测试好的最终结果；如在复现环境中重新执行推理，程序会生成 result.zip 作为当次运行产物。
3	README.md	代码说明文件，包含运行环境、配置说明、运行方法和输出产物说明
4	src/*	任务策略的源代码、配置文件、依赖列表
5	data/*	原始赛题文件

算法成绩

FlagOS 平台评测：74.95 分。

任务	说明	正确率	分数
1	计算列表中任意两个整数之间的最小绝对差	100%	12.5
2	统计给定句子中名词或动词的数量	80.4%	10.05
3	执行考拉兹猜想（Collatz Conjecture）单步计算	100%	12.5
4	将字符串列表按顺序拼接成完整字符串	100%	12.5
5	判断推文情感是否属于“悲伤”分类	83.1%	10.6
6	判断两个句子是否属于同一文本领域	78.8%	9.85
7	根据类别和线索生成 Jeopardy! 问答答案	52%	6.5
8	编写高效的 Triton GPU 算子内核	3.6%	0.45
SUM	总计	74.95%	74.95

架构设计

本系统采用分层解耦的设计思想，通过策略模式（Strategy Pattern）将通用的推理流程和任务逻辑分离。系统由以下层级组成，确保了从数据输入到结果产出的全流程稳定性。

入口与编排层

- `main.py`: 作为统一入口，负责环境检查、参数解析以及任务调度。

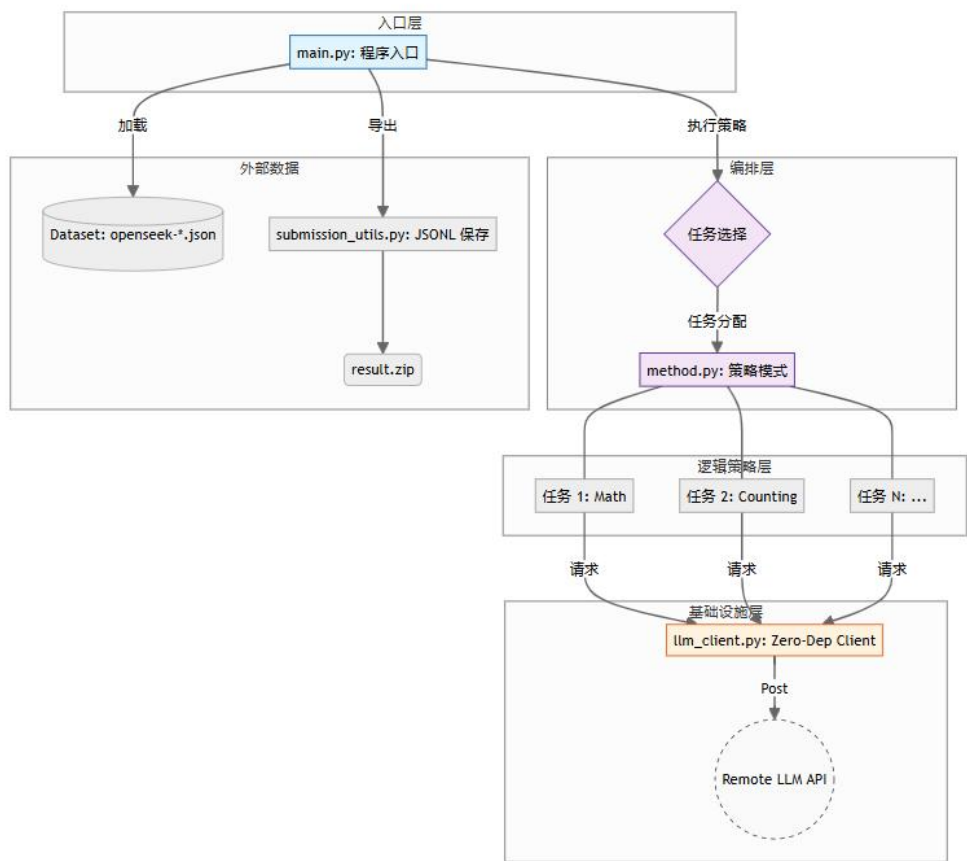
逻辑策略层

- `method.py`: 定义了所有策略，处理任务和策略的映射关系
- `strategy_*.py`: 每个任务拥有独立的提示词构建模板和后处理解析逻辑。

基础设施层

该层与模型部署解耦，可直接接入由 FlagScale 托管的 Qwen3-4B 服务：

- `llm_client.py`: 提供了一个轻量客户端，负责读取配置、拼接 OpenAI 兼容接口请求并附带 API Key。
- `submission_utils.py`: 自动完成多任务结果的打包（ZIP），确保提交产物符合比赛格式规范。



长上下文下的持续交互机制

本方案在多个任务中引入了轻量级的持续交互机制，用于提升长上下文场景下的稳定性。这里的“持续交互”并不局限于传统聊天式多轮对话，而是指系统围绕同一输入样本进行分阶段协作。

具体体现为以下三类机制：

- 分阶段决策：对于任务 5、任务 6 这类语义判断问题，系统不会总是一步直接输出标签，而是先进行逻辑预判或候选领域识别，再进入最终判断阶段，以减少长输入下的直接误判。
- 生成后校验：对于任务 1/3/4 的可验证程序策略，以及任务 8 的代码生成策略，系统会先生成候选程序，再结合样例或静态规则进行验证，必要时进入下一轮修复或重试。
- 动态上下文组织：对于任务 5、任务 7、任务 8，系统并非机械使用固定示例，而是根据输入内容动态筛选更相关的 Few-shot 或关键信息片段，把有限上下文优先分配给更有判别价值的内容。

策略：可验证程序（任务 1/3/4）

针对数学运算（任务 1）与逻辑规则推理（任务 3, 4），本方案并未采用传统的“逐样本推理”模式。此类任务逻辑极其确定，但输入序列较长，模型直接输出答案容易产生算术错误。

核心思路

本策略借鉴了工具调用的思想，将 LLM 的角色从“答案预测器”转变为“代码生成器”：

- 逻辑转换：引导模型观察 Few-shot 示例，生成一段通用的 Python 代码。
- 多轮验证：生成的代码不会被直接采用，系统会提取 Task 中自带的示例作为单元测试（Unit Test）。
- 自我迭代：如果代码在测试用例上失败，系统会多次重试，直到得到能通过样例检查的程序。
- 一次生成，多次执行：一旦验证通过，同一段程序即可复用于当前任务的全部测试样本。

诱导模板设计

为引导 Qwen3-4B 生成高质量且可执行的代码，设计了包含三个固定用例的思维诱导模板：

序号	用例说明	逻辑目标
1	比较两个数值，取最大值	展示如何处理数值计算与 json.loads 解析
2	计算一个数的阶乘	展示如何导入 math 库进行数学计算
3	取出序列中的第一个元素	展示如何利用 ast.literal_eval 解析输入序列

代码示例

使用未闭合 xml 标签，诱导模型输出：

```
# 模型在看到以下示例后，会自动补全 Task 4 的逻辑
<case3>
  <description>Extract the first element from a list.</description>
  <code>def solution(input: str) -> str:
    import ast
    data = ast.literal_eval(input)
    return str(data[0])</code>
</case3>
<case4>
  <description>{当前任务描述}</description>
  <examples>{任务自带的 Few-shot}</examples>
  <code>def solution(input: str) -> str:
```

策略：扫描统计（任务 2）

任务 2 要求统计长句中的语法成分（名词或实义动词）。对 4B 级别模型而言，难点不在于词性识别，而在于长句条件下容易出现计数遗漏、重复或对助动词的误判。

- 本策略采用“显式扫描”替代“隐式计数”方案，将任务拆解为逐词分析过程：
- 输入解析：预先提取目标句子与统计对象（Noun/Verb），排除 Prompt 中任务说明的干扰。
 - 结构化诱导：通过强制输出 <reasoning> 标签，诱导模型逐词列出语法成分及其所属类别。
 - 结果约束：最终数量必须包裹在 <answer> 标签中，并通过正则匹配进行健壮提取，确保输出格式合规。

策略：两段式逻辑分析（任务 5）

任务 5 是推文情感（悲伤）检测。4B 模型在处理这类任务时，容易被推文中的表情符号、口语化表达或反讽语气误导，进而在“Sad / Not sad”的边界样本上出现偏差。

本方案采用了“两段式动态逻辑”策略：

- 逻辑预判：先让模型对目标推文进行一次极短的“逻辑预判”（找出推文产生情感的核心理由，如 loss, irony, excitement），不直接出答案。
- 动态检索：利用预判的关键词，从示例中动态筛选出逻辑最接近的若干个 Few-shot。
- 类别平衡：在动态示例筛选时，额外检查正负样本分布，避免候选示例过度偏向单一标签。
- 结果约束：在最终判断阶段，系统只保留“Sad”或“Not sad”两类合法输出。

策略：语义一致性校验（任务 6）

任务 6 需要判断两个句子是否属于同一预设领域。测试发现，Qwen3-4B 模型对“领域边界”的界定不清晰。

本方案采用“多候选匹配 + 一致性检查”的策略：

- 潜在领域提取：首先让模型分别列出 Sentence 1 和 Sentence 2 可能属于的所有潜在领域（允许输出多个候选），降低单一检测产生的误判。
- 一致性判定：如果两个句子的候选领域列表都包含了参考领域，则直接判定为 Y；如果任何一句明确排除了参考领域，则判定为 N。
- 不确定样本复核：对于不确定的样本，引入 LLM 进行重新复核，直接提取结果作为最终结论。

策略：动态示例提取（任务 7）

任务 7 是类似 Jeopardy 的百科问答，小模型在处理知识问答时，存在知识盲区。

本方案采用“动态示例提取”的策略：

- 动态示例检索：根据题目所属的 Category 关键词，动态提取同类范畴的多个示例
- 重试机制：系统会对输出结果进行长度校验。如果答案部分字符数明显异常，程序会自动重试，直到输出符合规范的极简答案。

策略：约束式代码生成与静态修复（任务 8）

任务 8 (Triton 代码生成) 是全场难度最高的任务，本方案在该任务上的得分较低，主要围绕‘提升可执行性’做了一些尝试：

- 输入压缩：任务描述通常较长，整段生成容易使模型出现代码结构失控，因此系统会优先保留与 kernel、wrapper、shape、mask、stride、block 等关键信息相关的内容，压缩无关描述。
- 静态校验：代码生成后，系统会基于 Python ast 对结果做静态扫描，检查语法错误、明显的未定义符号，优先过滤掉不可执行的输出。
- 修复重试：若检测到明显错误，系统会将错误信息连同原始规格一并反馈给模型，尝试进行一次面向问题的修复。

部署与合规说明

本方案严格按照赛事要求，基于 Qwen3-4B 模型开展 ICL 自动标注实验，未使用任何其他模型或外部数据。在模型部署层面，底层推理服务由 FlagScale 负责模型加载、运行与服务化管理；项目代码侧通过统一客户端调用推理接口，将任务策略实现与模型部署细节解耦。

为便于工程复现，系统采用 OpenAI 兼容接口形式访问推理服务，实际承载模型推理的底层框架仍为 FlagScale，满足赛事关于开发框架与复现一致性的要求。

结语

感谢主办方举办这次富有挑战性的竞赛活动，提供了规则严谨且极具趣味性的赛题。同时感谢各位老师赛前提供的详细技术说明和指导。在当前 AI 快速发展的时代，AI 工具极大地放大了个人的学习能力和创造潜力，希望每个有想法的人能够实现更多可能。